TITLE:

Implementation of read() and write() System Calls in Linux

OBJECTIVE:

- To comprehend the role of system calls within the Linux environment.
- To analyze the functional mechanics of read() and write().
- To develop a C-based implementation using these calls on Ubuntu.

RELATED THEORY:

System Call:

A system call serves as the programmatic interface between a user-level application and the Linux kernel. It acts as the bridge allowing user space to request services from kernel space.

read() System Call:

The read() function retrieves data from a designated file descriptor.

Syntax:

```
ssize_t read(int fd, void *buf, size_t count);
```

Parameters:

fd -> File descriptor (e.g., 0 for Standard Input)

buf -> Memory buffer to store the incoming data

count -> Total bytes to be read

write() System Call:

The write() function sends data to a designated file descriptor.

Syntax:

ssize_t write(int fd, const void *buf, size_t count);

Parameters:

fd -> File descriptor (e.g., 1 for Standard Output)

buf -> Pointer to the data being written

count -> Total bytes to be written

Program:

```
#include <unistd.h>
#include <fcntl.h>

int main() {
    char data[80];

    write(1, "Input text: ", 12);

    int bytes = read(0, data, 80);

    write(1, "Output: ", 8);
    write(1, data, bytes);

    return 0;
}
```

Output:

```
prasanna-pokharel@prasanna-pokharel:~/LabWork$ clang rw.c -o rw_p
prasanna-pokharel@prasanna-pokharel:~/LabWork$ ./rw_p
Input text: Ram
Output: Ram
prasanna-pokharel@prasanna-pokharel:~/LabWork$
```

The program was executed using the clang compiler

TITLE:

Study and Implementation of the open() System Call in Linux (Ubuntu)

OBJECTIVE:

• To gain a clear understanding of the open() system call's role.

• To learn the process of creating and accessing files through Linux system calls.

• To develop a C program that demonstrates the functionality of open().

RELATED THEORY:

open() System Call:

The open() system call is utilized to access an existing file or generate a new file within the filesystem.

Syntax:

int open(const char *pathname, int flags, mode_t mode);

Parameters:

• pathname: The specific name or file path to be accessed.

• flags: The access mode used to open the file.

• mode: The security permissions assigned to the file (used with O_CREAT).

Commonly Used Flags:

O_RDONLY : Opens the file for read-only access.

O_WRONLY : Opens the file for write-only access.

O_RDWR : Opens the file for both reading and writing.

O_CREAT : Generates the file if it is not already present.

O_APPEND : Adds new data to the end of the file.

O_TRUNC : Clears existing content if the file already exists.

Note: Multiple flags can be integrated using the bitwise OR operator (|).

Program:

```c
#include <unistd.h>
#include <fcntl.h>

int main() {
    char data[80];

    write(1, "Input text: ", 12);

    int bytes = read(0, data, 80);

    write(1, "Output: ", 8);
    write(1, data, bytes);

    return 0;
}
```

Output:

```
prasanna-pokharel@prasanna-pokharel:~\LabWork$ clang open.c -o open_p
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./open_p
Task completed successfully
prasanna-pokharel@prasanna-pokharel:~\LabWork$ cat output.txt
System Programming
```

Output.txt automatically created with text 'System Programming'

TITLE:

Creating Threads in Linux via the POSIX Threads Library

OBJECTIVE:

- To generate a new execution thread using the pthread_create() function.
- To analyze the concurrent behavior of the main thread and its child thread.
- To understand the synchronization role of the pthread_join() function.

RELATED THEORY:

Thread:

A thread represents the most basic unit of CPU utilization within a process. Often referred to as a "lightweight process," it enables concurrent execution. While multiple threads in a single process share the same code segment, data, and resources, each maintains its own unique stack, registers, and program counter.

Core Functions:

pthread_create() : Initializes and starts a new thread.

pthread_join() : Synchronizes the main process, forcing it to wait for the thread's completion.

pthread_exit() : Manually ends the execution of the calling thread.

Required Header:

#include <pthread.h>

Program:

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void *worker_logic(void *arg);

int x, y;

int main() {
    pthread_t my_tid;

    pthread_create(&my_tid, NULL, worker_logic, NULL);

    pthread_join(my_tid, NULL);

    printf("Main execution started\n");

    for(y = 50; y < 55; y++) {
        printf("%d\n", y);
        sleep(1);
    }

    return 0;
}

void *worker_logic(void *arg) {
    printf("Thread execution started\n");

    for(x = 10; x < 15; x++) {
        printf("%d\n", x);
        sleep(1);
    }
```

```
    return NULL;
}
```

In above program,
pthread_t my_tid: Declares thread variable.
pthread_create() :Creates a new thread that runs thread_function().
pthread_join(): Main program waits until thread finishes
sleep(1) is used to clearly observe execution order.

Output:

```
prasanna-pokharel@prasanna-pokharel:~/LabWork$ clang POSIX_Thread.c -o posix_p
prasanna-pokharel@prasanna-pokharel:~/LabWork$ ./posix_p
Thread execution started
10
11
12
13
14
Main execution started
50
51
52
53
54
```

Here, thread printed numbers 10,11,12,13,14 and 15
Main program printed 50, 51, 52, 53 and 55

TITLE:
Analysis and Implementation of the fork() System Call for Process Creation in Linux

OBJECTIVE:
• To comprehend the fundamentals of process management in a Linux environment.
• To generate a new child process utilizing the fork() system call.
• To distinguish between the execution flows of parent and child processes.

RELATED THEORY:

Process:
A process is essentially a program during its execution phase. Every process is allocated its own memory space and is identified by a unique Process ID (PID).

Child Process:
A child process is a secondary process spawned by a "parent" process through the fork() system call. While it inherits many attributes from the parent, it remains an independent entity with its own PID. This mechanism ensures process isolation and enables concurrent task handling.

fork() System Call:
The fork() call is the primary method used to duplicate the calling process to create a new child.

Required Header:
#include <unistd.h>

Syntax:
pid_t fork(void);

Return Values of fork():

< 0 : Indicates that process creation was unsuccessful.
0 : Returned specifically within the newly created child process.
> 0 : Returned to the parent process (this value is the PID of the child).

Program:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t process_id;

    process_id = fork();

    if (process_id < 0) {
        printf("Error: Forking failed\n");
        return 1;
    }
    else if (process_id == 0) {
        printf("Current: Child Process\n");
        printf("ID: %d\n", getpid());
    }
    else {
        printf("Current: Parent Process\n");
        printf("Parent ID: %d\n", getpid());
        printf("Child ID created: %d\n", process_id);
    }

    return 0;
}
```

Here,
fork() creates a new child process.
After fork(), both parent and child execute the same program.
In the child process, fork() returns 0.
In the parent process, fork() returns the child's PID.
getpid() is used to display process ID.

Output:

```
prasanna-pokharel@prasanna-pokharel:~\LabWork$ clang fork.c -o fork_p
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./fork.c
bash: ./fork.c: Permission denied
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./fork_p
Current: Parent Process
Parent ID: 1355
Child ID created: 1356
Current: Child Process
ID: 1356
```

TITLE:
Study and Implementation of the wait() System Call in Linux

OBJECTIVE:
• To understand the functional purpose of the wait() system call.
• To achieve process synchronization between a parent and its child.
• To eliminate the formation of "zombie processes" by reposing child exit statuses.

RELATED THEORY:

wait() System Call:
The wait() system call allows a parent process to suspend its execution until one of its child processes has completed.

This mechanism is vital for ensuring that processes execute in a synchronized order and that the parent can retrieve the results of the child's work.

Required Headers:
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

Syntax:
pid_t wait(int *status);

Operation of wait():
1. Activation: Once wait() is invoked, the parent process enters a "wait" state.
2. Termination: The parent remains paused until a child process terminates.
3. Reporting: The system returns the PID of the finished child to the parent.
4. Exit Status: Information regarding how the child ended is saved in the 'status' variable.

Program:
```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```c
int main() {
    pid_t child_p;

    child_p = fork();

    if (child_p < 0) {
        printf("Error: Fork failed\n");
        return 1;
    }
    else if (child_p == 0) {
        printf("Child starts execution\n");
        sleep(2);
        printf("Child execution done\n");
    }
    else {
        wait(NULL);
        printf("Parent resumed after child\n");
    }

    return 0;
}
```

Output:

```
prasanna-pokharel@prasanna-pokharel:~\LabWork$ clang wait.c -o wait_p
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./wait_p
Child starts execution
Child execution done
Parent resumed after child
```

The wait() system call synchronized parent and child processes and prevented zombie process creation.

TITLE

Study and Implementation of Passing Parameters to Threads using POSIX Threads

OBJECTIVE

- To understand how arguments are passed to a thread.
- To create threads with user-defined parameters.
- To observe thread execution using passed values.

RELATED THEORY
Passing Arguments to Threads:
In POSIX threads, arguments are passed to a thread function using the fourth parameter of:

pthread_create(&thread_id, NULL, function_name, argument);
The argument must be passed as a void *.

Inside the thread function, it must be typecast to the original data type.

Thread Function Format:
void* thread_function(void* arg);

Program:

```
#include <stdio.h>
#include <pthread.h>

void* display_val(void* arg) {
    int received = *(int*)arg;

    printf("Value in thread: %d\n", received);

    return NULL;
}

int main() {
    pthread_t thread_id;
```

```
    int my_number = 55;

    pthread_create(&thread_id, NULL, display_val,
&my_number);

    pthread_join(thread_id, NULL);

    return 0;
}
```

Output:

```
prasanna-pokharel@prasanna-pokharel:~\LabWork$ clang pass_pm.c -o pm_p
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./pm_p
Value in thread: 55
```

TITLE:
Demonstration of Race Conditions in C via POSIX Threads

OBJECTIVE:
• To develop a C-based program that illustrates the occurrence of a race condition.
• To analyze how multiple threads interact with a shared resource in a Linux environment.

RELATED THEORY:

Race Condition:
A race condition is a phenomenon that occurs when multiple threads or processes attempt to access and manipulate the same shared data simultaneously. In such scenarios, the final outcome of the program becomes non-deterministic, as it depends entirely on the unpredictable sequence or timing of thread execution.

This issue is a common challenge in concurrent programming, appearing frequently when using:
1. Multi-threading (e.g., pthread library)
2. Process-based concurrency (e.g., fork system call)

Critical Section:
The specific part of the code where the shared resource is accessed is known as the "critical section." Without proper synchronization (like Mutexes or Semaphores), threads may "race" to modify the data, leading to inconsistent or incorrect results.

Program:
```
#include <stdio.h>
#include <pthread.h>
```

```c
int shared_total = 0;

void* add_task(void* arg) {
    for(int j = 0; j < 50000; j++) {
        shared_total++;
    }
    return NULL;
}

void* sub_task(void* arg) {
    for(int j = 0; j < 50000; j++) {
        shared_total--;
    }
    return NULL;
}

int main() {
    pthread_t thread_a, thread_b;

    pthread_create(&thread_a, NULL, add_task, NULL);
    pthread_create(&thread_b, NULL, sub_task, NULL);

    pthread_join(thread_a, NULL);
    pthread_join(thread_b, NULL);

    printf("Resulting value: %d\n", shared_total);

    return 0;
}
```

Output:

```
prasanna-pokharel@prasanna-pokharel:~\LabWork$ clang race_condition.c -o race_p
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./race_p
Resulting value: -3223
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./race_p
Resulting value: 24230
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./race_p
Resulting value: -3590
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./race_p
Resulting value: -36000
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./race_p
Resulting value: 0
```

Here, the final output varies every time because both threads access the shared variable counter simultaneously.

TITLE:
Process and Thread Synchronization Using Semaphores

OBJECTIVE:
• To analyze the nature of race conditions within programs that utilize shared variables across multiple threads.
• To examine the function of semaphores as a fundamental synchronization primitive for maintaining mutual exclusion in critical sections.
• To implement practical synchronization in C using the POSIX functions sem_wait() and sem_post().

RELATED THEORY:

RACE CONDITIONS:
A race condition occurs when concurrent threads access and modify a shared resource, leading to a final outcome that is dependent on the non-deterministic scheduling of the OS. In the demonstrated program, a global "counter" is simultaneously incremented and decremented 100,000 times by different threads. Since these operations are not "atomic" at the hardware level—consisting of separate read, modify, and write steps—interleaved execution results in data inconsistency and unpredictable final values.

SEMAPHORES:
Introduced by Edsger Dijkstra in 1965, a semaphore is an integer-based synchronization tool. It manages access to shared resources through two primary atomic operations:

sem_wait() (Wait/P Operation):
This function decrements the semaphore's value. If the resulting value is less than zero, the executing thread is placed in a blocked state until the resource becomes available.

sem_post() (Signal/V Operation):
This function increments the semaphore's value. If any threads were previously blocked and waiting for the resource, the system unblocks one, allowing it to enter its critical section.

Required Header:
#include <semaphore.h>

Program:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

int global_val = 0;
sem_t my_sem;

void* plus_op(void* arg) {
    for(int k = 0; k < 75000; k++) {
        sem_wait(&my_sem);
        global_val++;
        sem_post(&my_sem);
    }
    return NULL;
}

void* minus_op(void* arg) {
    for(int k = 0; k < 75000; k++) {
        sem_wait(&my_sem);
        global_val--;
        sem_post(&my_sem);
    }
    return NULL;
}

int main() {
    sem_init(&my_sem, 0, 1);

    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, plus_op, NULL);
    pthread_create(&thread2, NULL, minus_op, NULL);

    pthread_join(thread1, NULL);
```

```
    pthread_join(thread2, NULL);

    printf("Total: %d\n", global_val);

    sem_destroy(&my_sem);
    return 0;
}
```

Output:

```
prasanna-pokharel@prasanna-pokharel:~\LabWork$ clang semaphore.c -o sem_p
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./sem_p
Total: 0
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./sem_p
Total: 0
```

This output of 0 is expected

TITLE:

Synchronization Using Mutex Locks

OBJECTIVE:

• To examine Mutex locks as a synchronization tool within the POSIX thread library for maintaining mutual exclusion in critical sections.

• To implement practical thread synchronization using pthread_mutex_lock() and pthread_mutex_unlock() in a C-based multithreaded environment.

RELATED THEORY:

Mutex Locks (Mutual Exclusion):

A Mutex is a specialized synchronization primitive from the pthread library used to safeguard critical sections. It ensures that only a single thread can interact with a shared resource at any given moment. Unlike general semaphores, a Mutex is strictly designed for mutual exclusion; the specific thread that acquires the lock is the only one authorized to release it.

Key Functionality:

• pthread_mutex_init() : Prepares the mutex object before thread execution begins.

• pthread_mutex_lock() : Requests ownership of the lock. If the lock is currently held, the calling thread pauses until the lock becomes available.

• pthread_mutex_unlock() : Releases the held lock, signaling the system to allow a waiting thread to enter.

• pthread_mutex_destroy() : Deallocates the mutex resources once synchronization is no longer required.

Resolving Race Conditions:

In a multithreaded context, operations like "counter++" or "counter--" are non-atomic, meaning they involve a three-step sequence: reading the value, modifying it, and writing it back. Without a Mutex, these steps can be interrupted by other threads, leading to "lost updates" and data corruption. By wrapping these operations in a Mutex lock, the code segment becomes an atomic-like unit, ensuring that no two threads can interleave their instructions within the critical section.

Program:

```
#include <stdio.h>
#include <pthread.h>

int shared_data = 0;
pthread_mutex_t m_lock;

void* task_up(void* arg) {
    for(int n = 0; n < 120000; n++) {
        pthread_mutex_lock(&m_lock);
        shared_data++;
        pthread_mutex_unlock(&m_lock);
    }
    return NULL;
}

void* task_down(void* arg) {
    for(int n = 0; n < 120000; n++) {
        pthread_mutex_lock(&m_lock);
        shared_data--;
        pthread_mutex_unlock(&m_lock);
    }
    return NULL;
}

int main() {
    pthread_mutex_init(&m_lock, NULL);
```

```
    pthread_t thread_1, thread_2;

    pthread_create(&thread_1, NULL, task_up, NULL);
    pthread_create(&thread_2, NULL, task_down, NULL);

    pthread_join(thread_1, NULL);
    pthread_join(thread_2, NULL);

    printf("Final total: %d\n", shared_data);

    pthread_mutex_destroy(&m_lock);
    return 0;
}
```

Output:

```
prasanna-pokharel@prasanna-pokharel:~\LabWork$ clang mutex_locks.c -o mutex_p
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./mutex_p
Final total: 0
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./mutex_p
Final total: 0
```

This output of 0 is expected

TITLE:

Inter-Process Communication (IPC) via Shared Memory

OBJECTIVE:

• To understand the necessity and mechanisms of Inter-Process Communication (IPC) for independent processes.

• To study the POSIX/System V shared memory API, specifically functions like shmget(), shmat(), and shmdt().

• To implement a C program that accesses a shared memory segment using a predefined key to retrieve stored data.

RELATED THEORY:

Inter-Process Communication (IPC):

IPC is a set of operating system mechanisms that enable separate processes to exchange data and synchronize actions. Because Linux ensures process isolation, processes cannot naturally access each other's memory. The OS facilitates this through pipes, message queues, sockets, and shared memory. Shared memory is considered the most efficient IPC method because it avoids the overhead of copying data between process address spaces; instead, multiple processes reference the same physical memory.

Shared Memory:

This mechanism involves mapping a specific memory segment into the address space of multiple processes. Once established, processes interact with this segment as they would with local memory, bypassing the need for further system calls during data exchange.

Primary Functions:

• shmget(key, size, flags) : Creates or finds a shared memory segment linked to a specific "key." It returns a shared memory identifier (shmid). In this implementation, a fixed key (1122) is used to locate an existing segment.

• shmat(shmid, NULL, 0) : Attaches the identified memory segment to the current process. It returns a pointer to the shared region. By passing NULL, the OS is allowed to select the attachment address automatically.

• shmdt(shared_memory) : Detaches the segment from the process's address space. Note that detaching does not delete the segment; it remains in the system.

• shmctl(shmid, IPC_RMID, NULL) : Deletes the shared memory segment from the system. This is typically called by the final process after communication is complete.

Required Header:

#include <sys/shm.h>

Program:

```
// sender program

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/shm.h>
#include <string.h>

int main() {
```

```
    void *shm_ptr;
    int mem_id;

    mem_id = shmget((key_t)5566, 2048, 0666 |
IPC_CREAT);

    printf("ID of allocated memory: %d\n", mem_id);

    shm_ptr = shmat(mem_id, NULL, 0);

    printf("Process attached at address: %p\n",
shm_ptr);

    printf("Write a message: ");
    fgets((char *)shm_ptr, 2048, stdin);

    printf("Data stored in memory: %s\n", (char
*)shm_ptr);

    shmdt(shm_ptr);

    return 0;
}

// reciever program

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/shm.h>
#include <string.h>

int main() {
    void *shm_ptr;
    int mem_id;

    mem_id = shmget((key_t)5566, 2048, 0666);

    printf("ID of allocated memory: %d\n", mem_id);
```

```
    shm_ptr = shmat(mem_id, NULL, 0);

    printf("Process attached at address: %p\n",
shm_ptr);

    printf("Data fetched: %s\n", (char *)shm_ptr);

    shmdt(shm_ptr);

    shmctl(mem_id, IPC_RMID, NULL);

    printf("Cleanup completed.\n");

    return 0;
}
```

Output:

```
prasanna-pokharel@prasanna-pokharel:~\LabWork$ clang sender.c -o send_p
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./send_p
ID of allocated memory: 0
Process attached at address: 0x7f8561994000
Write a message: hello!
Data stored in memory: hello!

prasanna-pokharel@prasanna-pokharel:~\LabWork$ clang reciever.c -o get_p
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./get_p
ID of allocated memory: 0
Process attached at address: 0x7f985c380000
Data fetched: hello!

Cleanup completed.
```

TITLE:

The Dining Philosophers Problem: Synchronization and Resource Management

OBJECTIVE:

• To analyze the classical Dining Philosophers problem as a model for resource sharing and synchronization in concurrent systems.

• To investigate the risks of deadlock, starvation, and race conditions when multiple processes compete for limited shared resources.

• To implement a synchronized, deadlock-free solution in C using the semaphore library.

RELATED THEORY:

The Dining Philosophers Problem:

Proposed by Edsger Dijkstra in 1965, this problem features five philosophers seated at a round table. Each philosopher alternates between thinking and eating. A single bowl of food is in the center, and one chopstick is placed between each adjacent philosopher (five total). To eat, a philosopher must acquire both the chopstick to their left and the one to their right. After finishing, they return both chopsticks and go back to thinking.

The Synchronization Challenges:

Because the chopsticks are shared resources, several synchronization issues can occur:

1. Deadlock: This occurs if every philosopher simultaneously picks up their left chopstick. In this state, no one can acquire their right chopstick, and everyone waits indefinitely.

2. Starvation: This happens if a philosopher is perpetually blocked from eating because their neighbors are constantly alternating their turns at the table.

3. Race Condition: Without proper locking, two adjacent philosophers might attempt to grab the same chopstick at the exact same moment.

Semaphore-Based Solution:

In a C implementation, each chopstick is treated as a binary semaphore initialized to 1.

• Before eating: A philosopher executes sem_wait() on the required chopstick semaphores.

• After eating: They execute sem_post() to release the chopsticks for others.

To prevent the "Circular Wait" that leads to deadlock, a common strategy is to make one philosopher (the "odd man out") pick up their chopsticks in a different order than the others (e.g., right then left, instead of left then right).

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define TOTAL_GUESTS 6
```

```c
#define THINK_TIME 1
#define EAT_TIME 1

sem_t fork_locks[TOTAL_GUESTS];

void* execute_dining_cycle(void* arg)
{
    int guest_id = *(int*)arg;
    int left_fork = guest_id;
    int right_fork = (guest_id + 1) % TOTAL_GUESTS;

    printf("Guest %d is pondering life...\n",
guest_id);
    sleep(THINK_TIME);

    // Deadlock Prevention: The "Asymmetric" approach
    // If guest_id is the last one, they grab the right
fork first.
    if (guest_id == TOTAL_GUESTS - 1)
    {
        sem_wait(&fork_locks[right_fork]);
        printf("Guest %d secured fork %d (Right)\n",
guest_id, right_fork);

        sem_wait(&fork_locks[left_fork]);
        printf("Guest %d secured fork %d (Left)\n",
guest_id, left_fork);
    }
    else
    {
        sem_wait(&fork_locks[left_fork]);
        printf("Guest %d secured fork %d (Left)\n",
guest_id, left_fork);

        sem_wait(&fork_locks[right_fork]);
        printf("Guest %d secured fork %d (Right)\n",
guest_id, right_fork);
    }
```

```
    // Critical Section: Eating
    printf("Guest %d is currently eating.\n",
guest_id);
    sleep(EAT_TIME);

    // Release resources
    sem_post(&fork_locks[left_fork]);
    sem_post(&fork_locks[right_fork]);

    printf("Guest %d finished eating and released forks
%d & %d.\n", guest_id, left_fork, right_fork);

    return NULL;
}

int main()
{
    pthread_t guest_threads[TOTAL_GUESTS];
    int guest_indices[TOTAL_GUESTS];

    // Initialize semaphores
    for (int i = 0; i < TOTAL_GUESTS; i++) {
        sem_init(&fork_locks[i], 0, 1);
    }

    // Launch threads
    for (int i = 0; i < TOTAL_GUESTS; i++) {
        guest_indices[i] = i;
        if (pthread_create(&guest_threads[i], NULL,
execute_dining_cycle, &guest_indices[i]) != 0) {
            perror("Failed to create thread");
        }
    }

    // Synchronize threads
    for (int i = 0; i < TOTAL_GUESTS; i++) {
        pthread_join(guest_threads[i], NULL);
    }
```

```c
    // Cleanup
    for (int i = 0; i < TOTAL_GUESTS; i++) {
        sem_destroy(&fork_locks[i]);
    }

    printf("Dining session complete.\n");
    return 0;
}
```

Output:

```
prasanna-pokharel@prasanna-pokharel:~\LabWork$ clang dining_philosopher.c -o dine_p
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./dine_p
Guest 0 is pondering life...
Guest 1 is pondering life...
Guest 2 is pondering life...
Guest 3 is pondering life...
Guest 4 is pondering life...
Guest 5 is pondering life...
Guest 0 secured fork 0 (Left)
Guest 0 secured fork 1 (Right)
Guest 0 is currently eating.
Guest 2 secured fork 2 (Left)
Guest 2 secured fork 3 (Right)
Guest 2 is currently eating.
Guest 4 secured fork 4 (Left)
Guest 4 secured fork 5 (Right)
Guest 4 is currently eating.
Guest 0 finished eating and released forks 0 & 1.
Guest 2 finished eating and released forks 2 & 3.
Guest 5 secured fork 0 (Right)
Guest 4 finished eating and released forks 4 & 5.
Guest 1 secured fork 1 (Left)
Guest 1 secured fork 2 (Right)
Guest 1 is currently eating.
Guest 5 secured fork 5 (Left)
Guest 5 is currently eating.
Guest 3 secured fork 3 (Left)
Guest 3 secured fork 4 (Right)
Guest 3 is currently eating.
Guest 1 finished eating and released forks 1 & 2.
Guest 5 finished eating and released forks 5 & 0.
Guest 3 finished eating and released forks 3 & 4.
Dining session complete.
```

TITLE:

Deadlock Phenomena in Operating Systems

OBJECTIVE:

• To understand the nature of deadlocks and the specific environmental factors that cause them.

• To study the four essential conditions for deadlock: mutual exclusion, hold and wait, no preemption, and circular wait.

• To simulate a deadlock scenario in C by creating two threads that compete for the same shared resources.

• To observe how a "circular wait" state causes a multithreaded program to hang indefinitely.

• To analyze methods for detecting, preventing, and avoiding deadlocks in concurrent systems.

RELATED THEORY:

Deadlock:

A deadlock is a specific state in an operating system where a group of processes becomes permanently blocked. This happens because each process holds a resource that another process needs, while simultaneously waiting for a resource held by someone else. In this state, no process can move forward, release its current resources, or finish execution, bringing the system to a complete halt.

The Four Necessary Conditions for Deadlock:

For a deadlock to occur, all four of these conditions must be present simultaneously:

1. Mutual Exclusion: At least one resource is designated as non-shareable, meaning only one process can utilize it at any given time.

2. Hold and Wait: A process currently holding at least one resource is allowed to request and wait for additional resources that are being held by other processes.

3. No Preemption: The operating system cannot forcibly take a resource away from a process; resources must be released voluntarily by the process that owns them.

4. Circular Wait: A closed chain of processes exists where each process is waiting for a resource held by the next member in the chain, with the final process waiting for the first.

Program:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Renamed shared resources
pthread_mutex_t printer_lock;
pthread_mutex_t scanner_lock;

// Function for the first worker
void* office_worker_A(void* arg)
{
```

```
    printf("[Worker A] Attempting to secure the
Printer...\n");
    pthread_mutex_lock(&printer_lock);
    printf("[Worker A] Printer LOCKED.\n");

    // Hold the printer and wait, allowing Worker B to
grab the scanner
    sleep(1);

    printf("[Worker A] Now attempting to secure the
Scanner...\n");
    pthread_mutex_lock(&scanner_lock); // DEADLOCK:
Worker B has this

    printf("[Worker A] Success! Performing office
tasks.\n");

    pthread_mutex_unlock(&scanner_lock);
    pthread_mutex_unlock(&printer_lock);

    return NULL;
}

// Function for the second worker
void* office_worker_B(void* arg)
{
    printf("[Worker B] Attempting to secure the
Scanner...\n");
    pthread_mutex_lock(&scanner_lock);
    printf("[Worker B] Scanner LOCKED.\n");

    // Hold the scanner and wait, allowing Worker A to
grab the printer
    sleep(1);

    printf("[Worker B] Now attempting to secure the
Printer...\n");
    pthread_mutex_lock(&printer_lock); // DEADLOCK:
Worker A has this
```

```
    printf("[Worker B] Success! Performing office
tasks.\n");

    pthread_mutex_unlock(&printer_lock);
    pthread_mutex_unlock(&scanner_lock);

    return NULL;
}

int main()
{
    pthread_t thread_A, thread_B;

    // Initialize the mutexes
    pthread_mutex_init(&printer_lock, NULL);
    pthread_mutex_init(&scanner_lock, NULL);

    printf("Starting Office Simulation: Expecting
Circular Wait Deadlock...\n\n");

    pthread_create(&thread_A, NULL, office_worker_A,
NULL);
    pthread_create(&thread_B, NULL, office_worker_B,
NULL);

    // The program will hang here indefinitely
    pthread_join(thread_A, NULL);
    pthread_join(thread_B, NULL);

    // Cleanup (Technically never reached in a
deadlock)
    pthread_mutex_destroy(&printer_lock);
    pthread_mutex_destroy(&scanner_lock);

    return 0;
}
```

Output:

```
prasanna-pokharel@prasanna-pokharel:~\LabWork$ clang deadlock.c -o dead_p
prasanna-pokharel@prasanna-pokharel:~\LabWork$ ./dead_p
Starting Office Simulation: Expecting Circular Wait Deadlock...

[Worker A] Attempting to secure the Printer...
[Worker A] Printer LOCKED.
[Worker B] Attempting to secure the Scanner...
[Worker B] Scanner LOCKED.
[Worker B] Now attempting to secure the Printer...
[Worker A] Now attempting to secure the Scanner...



^[^[
```

Here, deadlock occurred and '^[' appeared on entering escape key

(Note: Enter Ctrl+c to retain terminal)

Why this happened (The 4 Coffman Conditions)

1. **Mutual Exclusion:** Only one worker can use the Printer/Scanner at a time.
2. **Hold and Wait:** Worker A holds the Printer while waiting for the Scanner.
3. **No Preemption:** The OS cannot forcibly take the Printer away from Worker A.
4. **Circular Wait:** A waits for B, and B waits for A.